Algoritmos

Alondra Ludueña

2026-04-18

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy parecido a MatLab que trabaja con variables que en general pueden verse como matrices.

```
a=38
A=40
a=40
b=60
c=b-a
c
```

```
## [1] 20
```

```
y=40
alpha<-30
```

Uso de datasets o base de datos internos de R

Exploración de Datos Preinstalados R incluye una librería de bases de datos para práctica. Para ver el catálogo de opciones disponibles, se utiliza el comando:

- data(): Abre una lista de todos los datasets internos que puedes cargar.

Visualización y Acceso a la Información Cuando trabajamos con un dataset específico, como cars (datos sobre velocidad y distancia de frenado), tenemos dos formas principales de interactuar con él:

- Llamada Global (cars): Escribir el nombre del objeto muestra la estructura completa de la base de datos (filas y columnas). Es útil para entender la relación entre las variables.
- Acceso por Columna (cars$speed): El símbolo $ actúa como un filtro o selector. Permite extraer una sola columna específica del dataset para trabajar con ella como un vector independiente.

```
summary(cars)
```

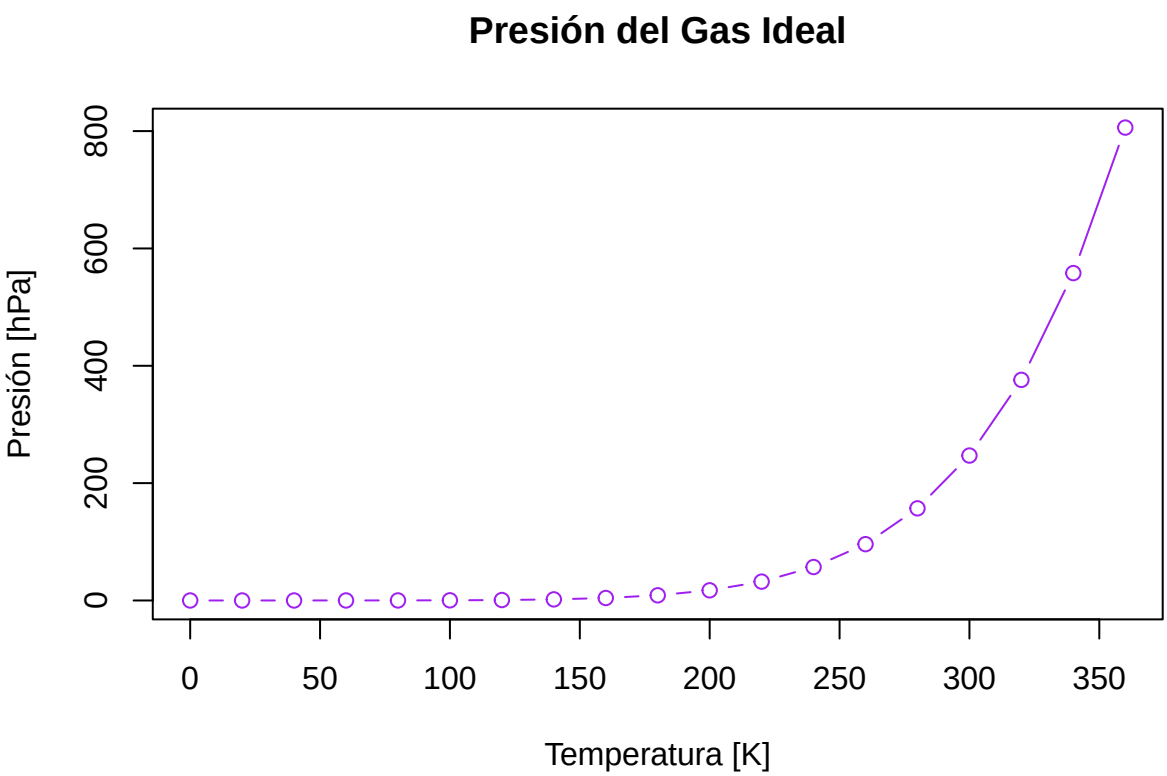
```
##      speed           dist
##  Min.   : 4.0   Min.   :  2.00
##  1st Qu.:12.0   1st Qu.: 26.00
##  Median :15.0   Median : 36.00
```

```
##  Mean   :15.4   Mean   : 42.98
##  3rd Qu.:19.0   3rd Qu.: 56.00
##  Max.   :25.0   Max.   :120.00
```

Including Plots

You can also embed plots, for example:

```
plot(pressure,main="Presión del Gas Ideal",ylab="Presión [hPa]",xlab="Temperatura [K]",type="b", col="pu
```



Comando plot

Es un comando que permite graficar cualquier fuente de datos que le asignamos, teniendo tambien la posibilidad de asignar nombre a las variables o hacer cambios al gráfico. En caso de duda de como utilizar el comando podemos escribir ##?plot## en la consola, lo cual abrirá una ventana que explicará a detalle el comando.

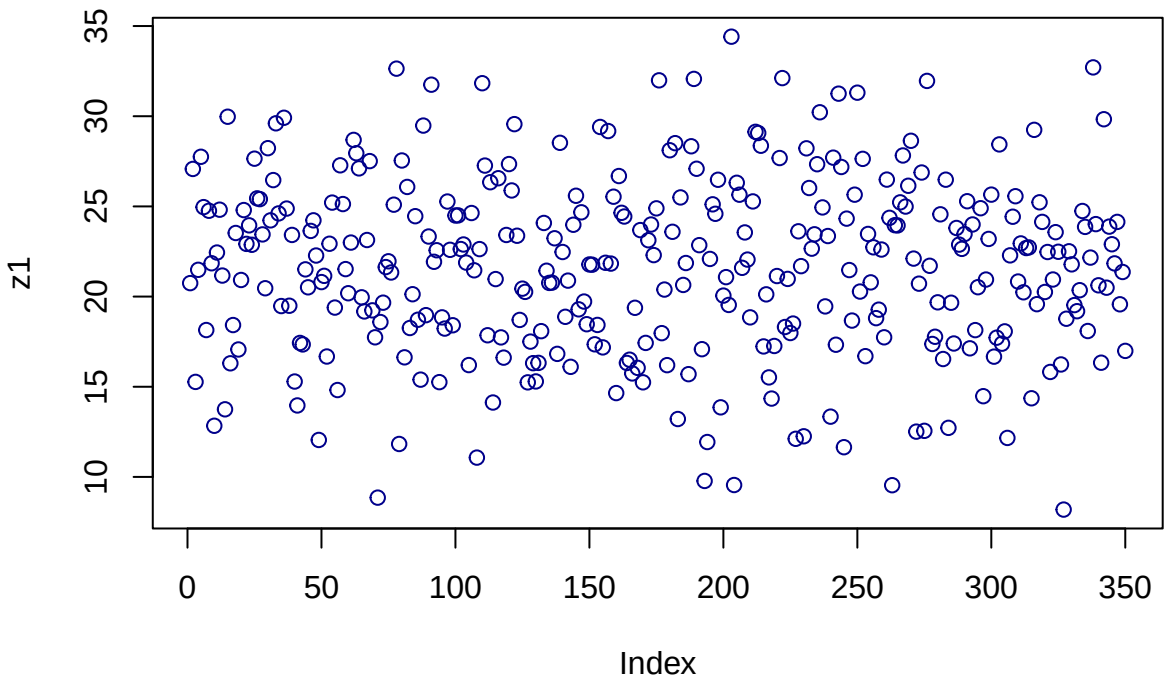
Funciones estadísticas

```
z1<-rnorm(350,21,5)
z1
```

```
##   [1] 20.745306 27.076328 15.268183 21.483782 27.751145 24.959574 18.144610
##   [8] 24.757918 21.846927 12.837211 22.440907 24.817363 21.165349 13.749468
##  [15] 29.971932 16.298920 18.421612 23.525830 17.066531 20.920525 24.796698
##  [22] 22.922003 23.949590 22.871244 27.646215 25.445334 25.402663 23.446910
##  [29] 20.461060 28.228880 24.225912 26.459482 29.606625 24.610501 19.473666
##  [36] 29.914498 24.880174 19.491677 23.417654 15.289179 13.962816 17.428542
##  [43] 17.345801 21.508097 20.512830 23.641079 24.226900 22.276035 12.050116
##  [50] 20.802829 21.149659 16.676985 22.928930 25.210585 19.389128 14.817683
```

```
##  [57] 27.277368 25.129206 21.523528 20.184932 22.993929 28.686615 27.949239
##  [64] 27.107582 19.959074 19.175745 23.136329 27.511228 19.235741 17.741208
##  [71]  8.851384 18.587601 19.661807 21.635749 21.956804 21.335471 25.095207
##  [78] 32.638423 11.827414 27.545616 16.630826 26.078991 18.257049 20.131054
##  [85] 24.457475 18.714857 15.394420 29.481571 18.969364 23.329443 31.748460
##  [92] 21.928268 22.568782 15.254938 18.847544 18.225348 25.269436 22.592591
##  [99] 18.409378 24.495056 24.501649 22.635687 22.888013 21.884064 16.201512
## [106] 24.633004 21.456631 11.071425 22.632285 31.830334 27.263735 17.854378
## [113] 26.340545 14.126377 20.970599 26.561907 17.736673 16.614400 23.416019
## [120] 27.347897 25.883468 29.560305 23.373107 18.705432 20.439066 20.260449
## [127] 15.243492 17.503633 16.308319 15.293802 16.322594 18.082454 24.080899
## [134] 21.432961 20.757425 20.772376 23.232943 16.823124 28.526302 22.471737
## [141] 18.877420 20.883807 16.104312 23.982780 25.588670 19.291380 24.673232
## [148] 19.724286 18.466425 21.780786 21.766725 17.353854 18.426974 29.403790
## [155] 17.182609 21.873365 29.177884 21.829131 25.537466 14.648828 26.684189
## [162] 24.644893 24.438789 16.319149 16.500372 15.739397 19.377778 16.047246
## [169] 23.687139 15.244739 17.430032 23.124389 23.995341 22.300255 24.892001
## [176] 31.989725 17.971906 20.388688 16.197426 28.113247 23.591466 28.507002
## [183] 13.210508 25.497562 20.644868 21.860663 15.697239 28.332913 32.069228
## [190] 27.086691 22.851167 17.082920  9.777745 11.934538 22.087132 25.115441
## [197] 24.589175 26.476556 13.858545 20.053485 21.078556 19.536351 34.410075
## [204]  9.551684 26.309547 25.648421 21.593703 23.555679 22.051226 18.847706
## [211] 25.275385 29.135082 29.067855 28.362791 17.235167 20.122871 15.519297
## [218] 14.345206 17.259890 21.138132 27.682081 32.114972 18.311990 20.979038
## [225] 17.979541 18.503670 12.112923 23.618637 21.687526 12.253063 28.219742
## [232] 26.018949 22.657444 23.453425 27.338307 30.218138 24.943028 19.454824
## [239] 23.356482 13.342510 27.702367 17.330034 31.251464 27.185056 11.647891
## [246] 24.322682 21.469755 18.669655 25.650264 31.306174 20.277599 27.635538
## [253] 16.696359 23.472401 20.787439 22.731912 18.803792 19.274149 22.609084
## [260] 17.741530 26.485654 24.366102  9.540996 23.958382 23.945329 25.224431
## [267] 27.824075 24.991007 26.144037 28.638633 22.106009 12.511280 20.715823
## [274] 26.879174 12.555456 31.956955 21.706751 17.371243 17.772154 19.680749
## [281] 24.560566 16.537594 26.481282 12.721272 19.658720 17.398136 23.802718
## [288] 22.881695 22.645374 23.464140 25.286913 17.130696 23.998024 18.139864
## [295] 20.519493 24.899221 14.476030 20.943473 23.196155 25.653673 16.668432
## [302] 17.720281 28.436790 17.388565 18.070146 12.161719 22.285021 24.426811
## [309] 25.564542 20.834017 22.947839 20.236957 22.691785 22.725949 14.360819
## [316] 29.244490 19.579606 25.227425 24.140885 20.254045 22.471463 15.819886
## [323] 20.946849 23.571613 22.483785 16.235885  8.189943 18.771288 22.510771
## [330] 21.787030 19.526591 19.188840 20.353983 24.744780 23.864265 18.089414
## [337] 22.161744 32.706544 24.018096 20.624706 16.333374 29.833609 20.492229
## [344] 23.879113 22.901054 21.839321 24.140659 19.571927 21.363976 16.987997
```

```r
plot(z1,col="darkblue") #comentario
```



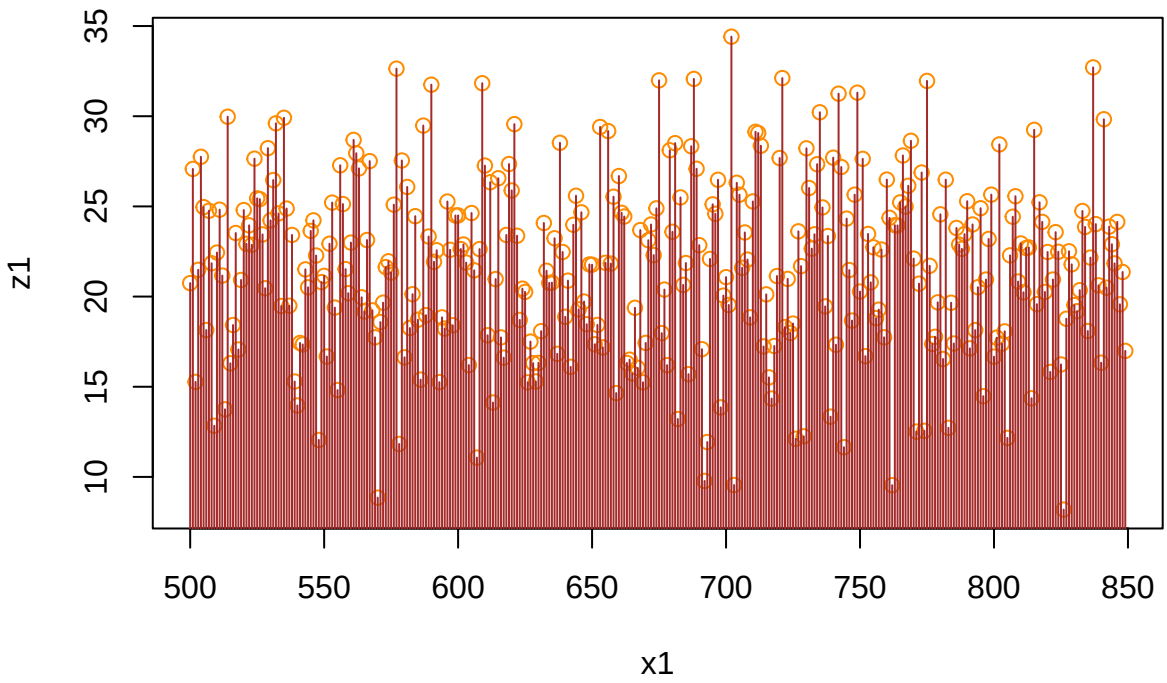

```r
w1<-length(z1)
w1
```

```
## [1] 350
```

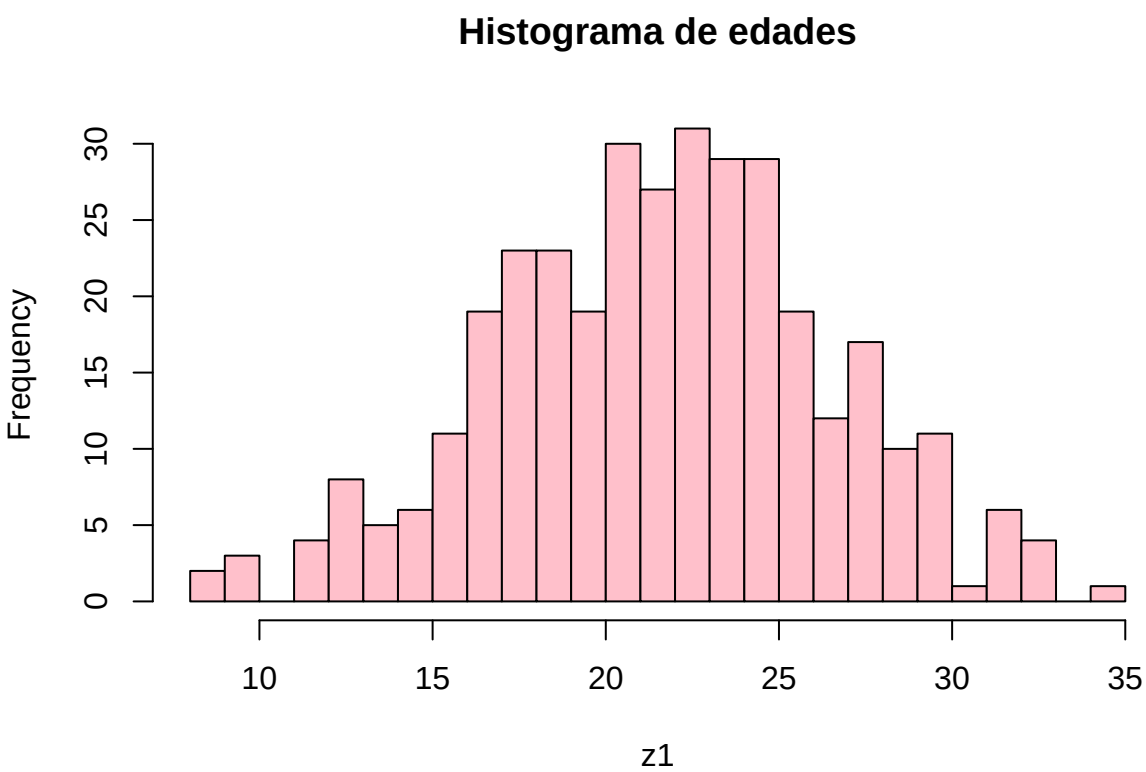
```r
x1<-(500:849)
x1
```

```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```r
plot(x1,z1,col="darkorange")
lines(x1,z1,col="brown",type="h")
```




```r
hist(z1, main="Histograma de edades",breaks=20,col="pink")
```

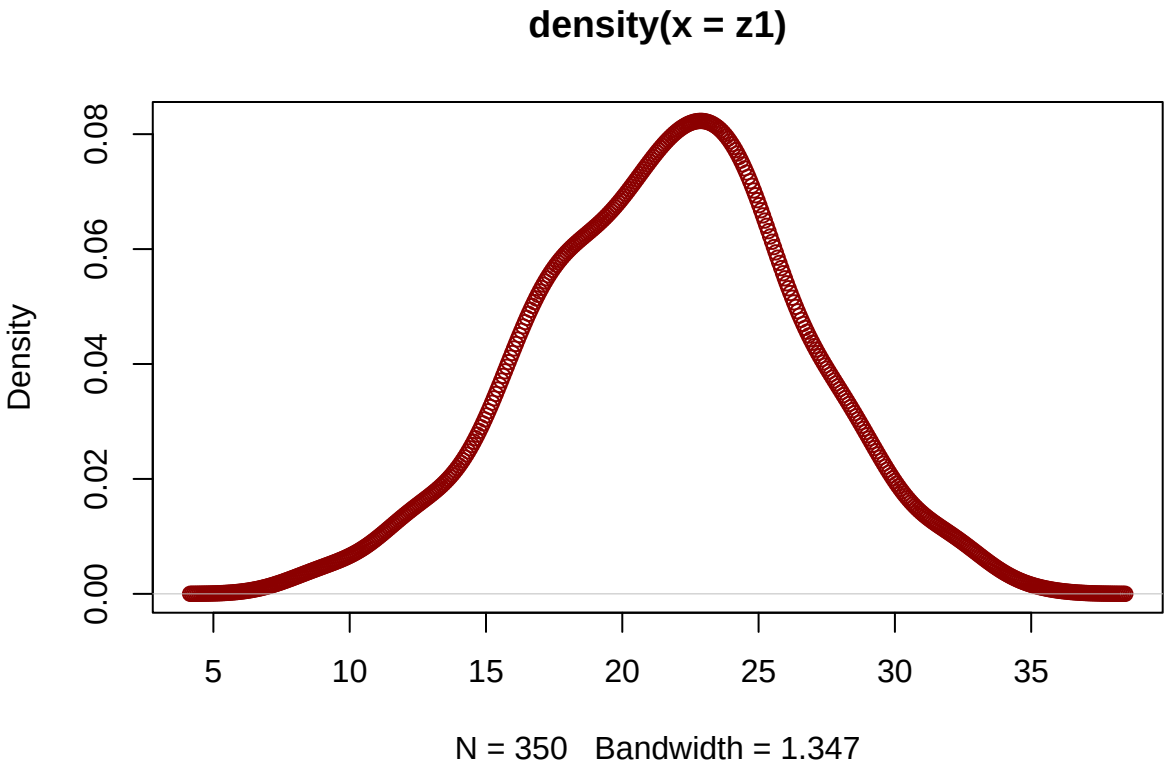



```r
density(z1)
```

```
##
## Call:
##  density.default(x = z1)
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.347
```

```
##
##        x                y
##  Min.   : 4.148   Min.   :9.651e-06
##  1st Qu.:12.724   1st Qu.:3.491e-03
##  Median :21.300   Median :1.775e-02
##  Mean   :21.300   Mean   :2.909e-02
##  3rd Qu.:29.876   3rd Qu.:5.550e-02
##  Max.   :38.452   Max.   :8.231e-02
```

```
plot(density(z1),col="darkred",type="b")
```



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Ejercicios de algoritmos

Las últimas 3 cifras de mi DNI son 848

Ejercicio 1 Crear una variable que tenga ese número

```
dni=848
```

Ejercicio 2 Crear un vector que tenga todos los números desde el 1 hasta el 848 enteros

```
secuencia_dni<-(1:dni)
secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
```

```
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558
## [559] 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576
## [577] 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594
## [595] 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612
## [613] 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630
## [631] 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648
## [649] 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666
## [667] 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684
## [685] 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702
## [703] 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720
## [721] 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738
## [739] 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756
## [757] 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774
## [775] 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792
## [793] 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810
## [811] 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828
## [829] 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846
## [847] 847 848
```

Ejercicio 3 Calcular la suma de todos los valores del vector secuencia_dni

```r
sum(1:length(secuencia_dni))
```

```
## [1] 359976
```

Ejercicio 4 Repetir el ejercicio anterior en Phyton

```
secuencia_dni=range(1,848)
total=sum(secuencia_dni)
print(total)
```

```
## 359128
```

Ejercicio 5 Repetir el ejercicio anterior con Julia. No se programó para no ocupar memoria.

Ejercicio 6 Determinar CPU time used.

```r
require(stats)
system.time(for(i in 1:100) mad(runif(1000)))
```

```
##    user  system elapsed
##   0.012   0.001   0.012
```

```r
require(stats)
system.time(for(i in 1:length(secuencia_dni)){})
```

```
##    user  system elapsed
##   0.001   0.000   0.002
```

```r
total<-0
valorfinal<-length(secuencia_dni)
for (i in 1:valorfinal){
total<-total+secuencia_dni[i]
}
print(total)
```

```
## [1] 359976
```

```r
sleep_for_a_minute <- function() { Sys.sleep(14) }
start_time <- Sys.time()
sleep_for_a_minute()
end_time <- Sys.time()
end_time- start_time
```

```
## Time difference of 14.01544 secs
```

Ejercicio 7 ¿Cuánto tarda en correr el código del ejercicio 2? con system.time

```r
tiempo <- system.time({
 sum(1:length(secuencia_dni))
})
print(tiempo)
```

```
##    user  system elapsed
##       0       0       0
```

Bibliotecta tictoc

Esto de usar una biblioteca es llamar u cargar una procedimientos que generará comando nuevos en R. Como ya fue comentado, cargar una biblioteca implica ejecutar el comando install.packages() o usar en r-studio el menú de Herramientas y Luego Instalar paquetes. Las funciones tic y toc son de la misma biblioteca de Octave/Matlab y se usan de la misma manera para la evaluacion comparativa que el tiempo de sistema recién demostrado. Sin embargo, tictoc agrega mucha más comodidad al usuario y armonía al conjunto.

```r
library(tictoc)
tic("sleeping")
A<-20
print("dormire una siestita...")
```

```
## [1] "dormire una siestita..."
```

```r
## [1] "dormire una siestita..."
Sys.sleep(2)
print("...suena el despertador")
```

```
## [1] "...suena el despertador"
```

```r
## [1] "...suena el despertador"
toc()
```

```
## sleeping: 2.005 sec elapsed
```

Biblioteca rbenchmark

La documentación de la función benchmark del paquete rbenchmark R lo describe como un simple contenedor alrededor de system.time. Sin embargo, agrega mucha conveniencia en comparación con las llamadas simples a system.time. Por ejemplo, requiere solo una llamada de referencia para cronometrar múltiples repeticiones de múltiples expresiones. Además, los resultados devueltos se organizan convenientemente en un marco de datos.

Implementación de una serie Fibonachi o Fibonacci

En matemáticas, la sucesión o serie de Fibonacci es la siguiente sucesión infinita de números naturales:0,1,1,2,3,5,8 ... 89,144,233 ... La sucesión comienza con los números 0 y 1,2 a partir de estos, cada término es la suma de los dos anteriores, es la relación de recurrencia que la define. A los elementos de esta sucesión se les llama números de Fibonacci. Esta sucesión fue descrita en Europa por Leonardo de Pisa, matemático italiano del siglo XIII también conocido como Fibonacci. Tiene numerosas aplicaciones en ciencias de la computación, matemática y teoría de juegos. Original de la Biblioteca Universidad de Florencia. Liber Abachi- Autor Fibonacci

Ejercicio 8 ¿Cuantas iteraciones se necesitan para generar un número de la serie mayor que 20 ? Además mostrar la serie de Fibonacci hasta el número obtenido anteriormente.

```r
# Inicialización
a <- 0
b <- 1
serie <- c(a, b)  # guardamos la serie
contador <- 2     # ya tenemos 2 términos
# Generar hasta superar 20
while (b <= 20) {
siguiente <- a + b
a <- b
b <- siguiente
serie <- c(serie, b)  # agregamos a la serie
contador <- contador + 1
}
# Resultados
cat("Cantidad de iteraciones:", contador, "\n")
```

```
## Cantidad de iteraciones: 9
```

```r
cat("Primer número mayor a 20:", b, "\n")
```

```
## Primer número mayor a 20: 21
```

```r
cat("Serie de Fibonacci hasta ese número:\n")
```

```
## Serie de Fibonacci hasta ese número:
```

```r
print(serie)
```

```
## [1]  0  1  1  2  3  5  8 13 21
```

Ejercicio 9 Compara la performance de ordenación del método burbuja vs el método sort de R. Usar método microbenchmark para una muestra de tamaño 20.000

```r
#### Instalar paquete
if (!require(microbenchmark)) {
install.packages("microbenchmark")
library(microbenchmark)
}
```

```
## Loading required package: microbenchmark
```

```r
#### Generar muestra
set.seed(123)
n <- 20000
vector <- sample(1:100000, n, replace = TRUE)
#### Implementación de ordenamiento burbuja
bubble_sort <- function(x) {
n <- length(x)
for (i in 1:(n-1)) {
for (j in 1:(n-i)) {
if (x[j] > x[j+1]) {
temp <- x[j]
x[j] <- x[j+1]
x[j+1] <- temp
}
}
}
return(x)
}
#### Benchmark (OJO: burbuja es muy lento)
resultado <- microbenchmark(
burbuja = bubble_sort(vector),
sort_R = sort(vector),
times = 5
)
print(resultado)
```

```
## Unit: microseconds
##     expr         min          lq         mean       median          uq
##  burbuja 20462370.42 20485572.87 2.055981e+07 20579531.039 2.06263e+07
##   sort_R      592.32      617.48 7.432324e+02      713.551 7.42161e+02
##          max neval
##  20645294.90     5
##      1050.65     5
```

La penitencia de Newton

Comentan algunos historiadores que Newton tenía en la escuela primaria un profesor muy exigente que reclamaba constantemente la atención de los alumnos a lo que el estaba enseñando. Tan pronto alguien cometía un acto improcedente lo castigaba con una penitencia que consistía en traer para el día siguiente el resultado de la suma desde el número 1 hasta cierto número aleatoriamente elegido. Por supuesto esta tarea demandaría quedarse toda la noche haciendo las sumas parciales, tarea que el profesor ya había realizado

previamente y que tenía registrado en un voluminoso cuaderno que (al estilo tabla de logaritmos) cargaba religiosamente en su portafolios de cuero.

Ejercicio 10 Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta 1*1010 y verifica cuál de los dos es más eficiente.

```r
#Algoritmo 1: suma con for (iterativo)
n <- 1010
suma1 <- 0
for (i in 1:n){
suma1 <- suma1 + i
}
suma1
```

```
## [1] 510555
```

```r
#Complejidad:O(n)(suma uno por uno)
#Algoritmo 2: fórmula matemática (Gauss)
n <- 1010
suma2 <- n * (n + 1) / 2
suma2
```

```
## [1] 510555
```

```r
#Complejidad:O(1)(una sola operación)
#Comparación de eficiencia:medir tiempos con system.time():
# Método 1
system.time({
suma1 <- 0
for (i in 1:1010){
suma1 <- suma1 + i
}
})
```

```
##    user  system elapsed
##   0.002   0.000   0.002
```

```r
# Método 2
system.time({
suma2 <- 1010 * (1010 + 1) / 2
})
```

```
##    user  system elapsed
##       0       0       0
```

Se desarrollaron dos algoritmos para sumar desde 1 hasta 1*1010. El método iterativo presenta complejidad lineal, lo que lo hace ineficiente para valores grandes como 10.000.000.000. En cambio, el uso de la fórmula cerrada reduce la complejidad a constante, permitiendo obtener el resultado de forma inmediata. Por lo tanto, el segundo algoritmo es significativamente más eficiente